

Question 1

1.

```
(aiopsvenv) eshikanahata@eshikanahata-IdeaPad:~/Documents/nl_project/assignment2/q1$ docker run -d -p 8080:8000 --name naive-final spam-api:naive
sleep 8
echo "--- healthz ---"
curl -s http://localhost:8080/healthz
echo
echo "--- predict ---"
curl -s -X POST http://localhost:8080/predict -H "Content-Type: application/json" -d '{"text": "WIN a FREE iPhone now!"}'
echo
docker stop naive-final && docker rm naive-final
428819aaed9d062b8b611820d63b5898a9389d0779b31f09c4375c859a4f837c
--- healthz ---
{"status": "ok"}
--- predict ---
{"label": "spam"}
naive-final
naive-final
(aiopsvenv) eshikanahata@eshikanahata-IdeaPad:~/Documents/nl_project/assignment2/q1$
```

Image size:

- Content Size: 260 MB
- Disk Usage: 915 MB

2.

```
(aiopsvenv) eshikanahata@eshikanahata-IdeaPad:~/Documents/nl_project/assignment2/q1$ docker run -d -p 8080:8000 --name multi-final spam-api:multistage
sleep 3
docker ps -a --filter name=multi-final
docker logs multi-final
a1282e4564d79a71ce5cf70d323defed3a53c3fc804299a290d4515f9fe7aa16
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
a1282e4564d7   spam-api:multistage   "uvicorn app:app --h..."   3 seconds ago   Up 3 seconds   0.0.0.0:8080->8000/tcp, [::]:8080->8000/tcp   multi-final
INFO: Started server process [1]
INFO: Waiting for application startup.
(aiopsvenv) eshikanahata@eshikanahata-IdeaPad:~/Documents/nl_project/assignment2/q1$ sleep 8
docker logs multi-final
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
(aiopsvenv) eshikanahata@eshikanahata-IdeaPad:~/Documents/nl_project/assignment2/q1$ curl -s http://localhost:8080/healthz
echo
curl -s -X POST http://localhost:8080/predict -H "Content-Type: application/json" -d '{"text": "Hey, are we still meeting for lunch?"}'
echo
{"status": "ok"}
{"label": "ham"}
(aiopsvenv) eshikanahata@eshikanahata-IdeaPad:~/Documents/nl_project/assignment2/q1$ docker stop multi-final && docker rm multi-final
multi-final
multi-final
```

Image size:

- Content Size: 164 MB
- Disk Usage: 713 MB

Metric	Naive	Multi-stage	% Reduction
Content size	260 MB	164 MB	36.9%
Disk usage	915MB	713 MB	22.1%

- Both images share the exact same python:3.12-slim base, so this reduction is entirely attributable to build structure, not base-image choice. The naive image permanently retains spam_dataset.csv (the raw training data), generate_dataset.py and train.py (needed only once, at build time, never at runtime), and pip's downloaded wheel cache from installing pandas and scikit-learn. The multi-stage build discards the entire builder stage (dataset, scripts, and pip cache included), carrying forward only the installed packages plus model.joblib and app.py into a fresh final stage, which accounts for the 36.9% drop in Content Size and 22.1% drop in Disk Usage.

Question 2

Cache hit speedup:

With the api and cache services running via docker compose up, sending an identical request twice in succession produced:

Request	Result	time_total
1st (cache MISS)	{"label":"spam"}	0.010445s
2nd (cache HIT)	{"label":"spam"}	0.004376s

This is a 58% reduction in latency on the second call.

3. Docker Compose solves the problem of defining and running multiple related containers on a single host as one coordinated unit, here, the api and cache services, wired together on a private Compose-managed network where each container reaches the other by service name (cache) rather than a hardcoded IP, as shown by the api container connecting to cache:6379 at startup. A single Dockerfile/container has no way to express that two containers need to exist together and communicate. Compose's scope stops at single-host orchestration, one docker compose up, one machine, with no scheduling across multiple hosts, no automatic rescheduling onto healthy nodes if one fails, and no rolling updates.

Question 3

- Validating a 40-row CSV shard with a regex-based email check is lightweight and I/O-bound, so each pod was given requests.cpu: "500m" with limits.cpu set identically (requests = limits, avoiding oversubscription). Against the assignment's stated 4 total allocatable CPUs (2 nodes × 2 CPUs), this permits $4 / 0.5 = 8$ pods to run concurrently, exactly matching completions: 8. parallelism was therefore set to 8.

2. minikube was recreated as a 2-node cluster (minikube start --nodes=2 --cpus=2). Applying the Job produced all 8 pods Completed within the same 43-second window, 3 pods scheduled on minikube, 5 on minikube-m02. This split isn't perfectly even, which is expected scheduler behavior: pods are assigned as the scheduler processes the queue rather than guaranteed to balance, and with sub-minute-lived pods there's little time for rebalancing.
- 3.

Shard	Invalid rows
0	3
1	4
2	5
3	6
4	7
5	3
6	4
7	5
Total	37

A shared volume was avoided because minikube's default storage provisioner backs PersistentVolumeClaims with a single-node hostPath-based implementation, not designed for safe concurrent writes from 8 simultaneously running pods, and in a multi-node cluster a ReadWriteMany volume isn't available without extra infrastructure, it isn't a capability Kubernetes gives you by default. Reading results back through the Kubernetes API gives each pod's result in its own container's log stream regardless of which node it ran on.

4. No. parallelism cannot exceed completions (8) creating more concurrent pods than there are shards to process would have no effect, since only 8 pods will ever be created regardless of available CPU. At 500m per pod, 6 CPUs would support up to 12 concurrent pods, which already exceeds the 8 needed; the job was already running at full concurrency (parallelism: 8) on the 4-CPU cluster, so the extra 2 CPUs of headroom don't unlock any further speedup.

Question 4

2.

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
shard-validation-0-w9wd9	0/1	Completed	0	9m43s	10.244.1.2	minikube-m02	<none>	<none>
shard-validation-1-78w6b	0/1	Completed	0	9m43s	10.244.1.3	minikube-m02	<none>	<none>
shard-validation-2-w5ph8	0/1	Completed	0	9m43s	10.244.0.5	minikube	<none>	<none>
shard-validation-3-qlp55	0/1	Completed	0	9m43s	10.244.1.6	minikube-m02	<none>	<none>
shard-validation-4-fz9n4	0/1	Completed	0	9m43s	10.244.0.4	minikube	<none>	<none>
shard-validation-5-m929n	0/1	Completed	0	9m43s	10.244.0.3	minikube	<none>	<none>
shard-validation-6-z6lcl	0/1	Completed	0	9m43s	10.244.1.4	minikube-m02	<none>	<none>
shard-validation-7-hxnk9	0/1	Completed	0	9m43s	10.244.1.5	minikube-m02	<none>	<none>
spam-api-deployment-685bf9bcbb-dvm4t	1/1	Running	0	39s	10.244.0.6	minikube	<none>	<none>
spam-api-deployment-685bf9bcbb-th5d4	1/1	Running	0	39s	10.244.1.7	minikube-m02	<none>	<none>
--- Deleted one pod below ---								
NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
shard-validation-0-w9wd9	0/1	Completed	0	10m	10.244.1.2	minikube-m02	<none>	<none>
shard-validation-1-78w6b	0/1	Completed	0	10m	10.244.1.3	minikube-m02	<none>	<none>
shard-validation-2-w5ph8	0/1	Completed	0	10m	10.244.0.5	minikube	<none>	<none>
shard-validation-3-qlp55	0/1	Completed	0	10m	10.244.1.6	minikube-m02	<none>	<none>
shard-validation-4-fz9n4	0/1	Completed	0	10m	10.244.0.4	minikube	<none>	<none>
shard-validation-5-m929n	0/1	Completed	0	10m	10.244.0.3	minikube	<none>	<none>
shard-validation-6-z6lcl	0/1	Completed	0	10m	10.244.1.4	minikube-m02	<none>	<none>
shard-validation-7-hxnk9	0/1	Completed	0	10m	10.244.1.5	minikube-m02	<none>	<none>
spam-api-deployment-685bf9bcbb-dvm4t	1/1	Running	0	57s	10.244.0.6	minikube	<none>	<none>
spam-api-deployment-685bf9bcbb-th5d4	1/1	Running	0	57s	10.244.1.7	minikube-m02	<none>	<none>

The ReplicaSet controller is responsible for this. The Deployment delegates pod management to a ReplicaSet, which continuously watches the actual pod count against its desired replicas: 2 via the label selector (app: spam-api). When one pod was deleted, the ReplicaSet's control loop observed only 1 matching pod instead of the desired 2 and immediately created a new one to reconcile the difference.

3.

[illegible]

Had this application been deployed as a Job instead of a Deployment, there would be no rolling-update mechanism at all: a Job runs its pod(s) to completion once and stops, it has no concept of a desired running-replica count to reconcile against, no RollingUpdate strategy, and no readiness-gated traffic cutover. Updating the image would require deleting and recreating the Job entirely, causing a full outage for a serving workload.

4. A Deployment is built around maintaining a continuously-available desired state, it assumes the workload should never stop serving, so it reconciles pod count indefinitely (self-healing) and shifts traffic gradually between old and new versions without downtime (rolling update). A Job is built around the opposite assumption: the workload has a defined endpoint, and success means reaching completions exactly once, not staying available forever, which is why Q3's Indexed Job pods were correctly configured with restartPolicy: Never and no rolling-update logic at all. Each fits its workload shape: the spam-detection API is a long-running service where availability during change is the priority, while shard validation is finite batch work where the priority is that every shard gets processed exactly once and the Job then exits cleanly.